REPORT TECNICO

SOC Automation & Threat Intelligence Triage

Redatto da: *Nicolò Calì Cybersecurity Specialist*

Data: *11/03/2026*

Oggetto: Automazione triage di sicurezza, parsing log JSON e integrazione API Threat Intelligence.

Indice

1. Introduzione ---------- *Pag.2*
2. Sviluppo dello Strumento di Triage ---------- *Pag.3*
   2.1 Ingestion dei log e Gestione della Memoria ---------- *Pag.3*
   2.2 Parsing dei Dati ed Estrazione degli IoC ---------- *Pag.3*
   2.3 Arricchimento tramite API e Secure Coding ---------- *Pag.4*
   2.4 Interrogazione Automatica del Servizio di Threat Intelligence ---- *Pag.4*
   2.5 Elaborazione del Rischio (Threat Scoring) ---------- *Pag.4*
3. Alerting e Reportistica ---------- *Pag.5*
4. Ottimizzazione e Funzionalità Avanzate ---------- *Pag.6*
   4.1 Implementazione del Caching Locale ---------- *Pag.6*
   4.2 Validazione e Gestione delle Eccezioni ---------- *Pag.6*
   4.3 Notifiche Real-Time via protocollo SMTP ---------- *Pag.7*
5. Conclusioni ---------- *Pag.8*

1. Introduzione

Il presente report documenta lo sviluppo di uno strumento in Python progettato per automatizzare le operazioni di **triage** iniziale all'interno di un **Security Operations Center**.

L'esercitazione mira a risolvere le inefficienze legate all'analisi manuale dei log di rete generati da sistemi **SIEM** (come Splunk o Suricata).

Il progetto è suddiviso in fasi sequenziali che comprendono:

- Gestione sicura della memoria per l'ingestion di file di log di grandi dimensioni
- Estrazione e il parsing di Indicatori di Compromissione (**IoC**) in formato **JSON**
- Arricchimento dei dati tramite l'integrazione di un'**API** di Threat Intelligence esterna
- Generazione automatizzata di report e alert basati sul livello di rischio rilevato con conseguente invio di una mail di alert.

2. Sviluppo dello Strumento di Triage

2.1 Ingestion dei log e Gestione della Memoria

La prima fase di sviluppo si è concentrata sulla creazione di un meccanismo di lettura dei file log generati dall'IDS.

Considerando la natura "**Big Data**" dei log di sicurezza, che possono raggiungere dimensioni considerevoli, si è optato per un approccio "**Memory-safe I/O**". Lo script è stato progettato per evitare caricamenti massivi in RAM (prevenendo blocchi **OOM**) implementando una lettura del file riga per riga.

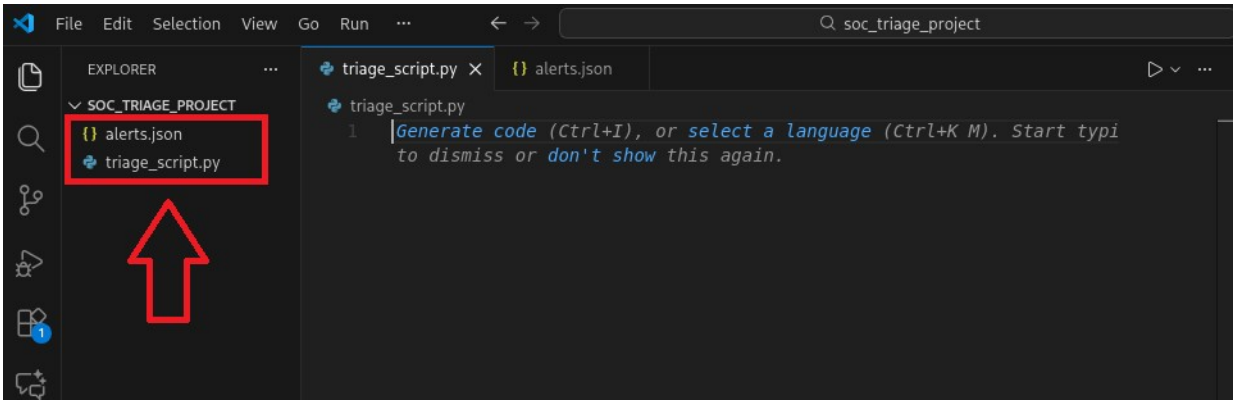

Fig. 1 - Setup iniziale dell'ambiente di sviluppo e creazione dei file di log di test.

2.2 Parsing dei Dati ed Estrazione degli IoC

Successivamente, l'attenzione si è spostata sul parsing strutturato dei log **JSON**. Tramite l'impiego delle librerie native di Python, i singoli eventi sono stati convertiti in dizionari, consentendo l'isolamento degli Indicatori di Compromissione (IoC), nello specifico l'indirizzo IP sorgente ("**src_ip**"). Per garantire la resilienza dello strumento di fronte a log malformati o incompleti, è stata implementata una rigorosa logica di **Exception Handling** (*costrutti try...except*). Questo approccio previene crash accidentali, assicurando la continuità dell'analisi sull'intero dataset.

Come evidenziato in *Fig. 2*, l'utilizzo del metodo `json.loads()` ha permesso la strutturazione dei dati grezzi.

L'implementazione di controlli di tipo `KeyError` e `JSONDecodeError` ha garantito l'esclusione di anomalie sintattiche (*log "sporchi" o privi del campo sorgente*) senza interrompere il flusso di esecuzione del programma principale.

Fig. 2 - Estrazione degli IoC ed esecuzione sicura tramite Exception Handling su log frammentati.

2.3 Arricchimento tramite API e Secure Coding

Per valutare l'effettiva pericolosità degli IP estratti, lo strumento è stato integrato con l'API REST di Threat Intelligence fornita da **VirusTotal**. Nel rispetto delle best practice di Secure Coding, si è evitato l'hardcoding delle credenziali all'interno dello script.

La API Key è stata gestita in modo sicuro tramite variabili d'ambiente caricate da un file esterno separato (.env), mitigando il rischio di esposizione accidentale.

2.4 Interrogazione Automatica del Servizio di Threat Intelligence

L'infrastruttura di rete dello strumento è stata sviluppata importando il modulo "requests", standard de facto per la gestione delle chiamate **HTTP** in Python.

Lo script estrapola dinamicamente gli indirizzi IP e costruisce query mirate verso gli endpoint REST di VirusTotal. Le credenziali di autenticazione sono state passate in modo sicuro tramite gli HTTP Headers. La validazione del canale di comunicazione è stata confermata dalla ricezione di codici di stato HTTP 200 (OK), indicando una corretta interazione server-to-server.

```
┌──(venv)─(kali㉿kali)-[~/Desktop/soc_triage_project]
└─$ python triage_script.py

IP da analizzare: 192.168.1.100
Stato connessione per 192.168.1.100: 200
```

Fig. 3 – Risposta della richiesta HTTP da parte di VirusTotal (200: OK)

2.5 Elaborazione del Rischio (Threat Scoring)

A seguito della corretta interrogazione dell'API, lo script è stato istruito per effettuare il parsing del payload JSON di risposta.

Data la complessità strutturale dei dati forniti da **VirusTotal**, si è resa necessaria una navigazione mirata all'interno dei nodi del dizionario risultante (*data* > *attributes* > *last_analysis_stats*) al fine di isolare la metrica "malicious".

Questo valore intero rappresenta il numero di engine di sicurezza globali che classificano il target come minaccia attiva, fornendo una base quantitativa essenziale per le successive fasi di alerting.

```
┌──(venv)─(kali㉿kali)-[~/Desktop/soc_triage_project]
└─$ python triage_script.py

IP da analizzare: 192.168.1.100
Rilevamenti malevoli per 192.168.1.100: 0
```

Fig. 4 - Parsing della risposta API e isolamento del Threat Score (rilevamenti malevoli).

3. Alerting e Reportistica

L'ultima fase del progetto ha riguardato l'operazionalizzazione dei dati raccolti. È stata implementata una logica condizionale (if...else) che valuta il Threat Score restituito dall'API: se il numero di rilevamenti malevoli risulta superiore a zero, lo script genera un alert critico a schermo. Parallelamente, l'esito di ogni singola analisi viene tracciato e storicizzato all'interno di un file di testo dedicato (report_triage.txt). L'apertura del file in modalità "append" garantisce la conservazione dello storico, fornendo al team SOC un artefatto leggibile e pronto per le successive fasi di Incident Response.

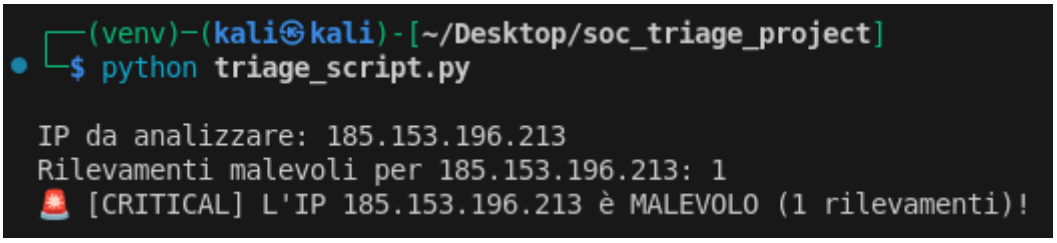

Fig. 5 - Rilevamento di un indicatore malevolo e generazione automatica dell'alert critico.

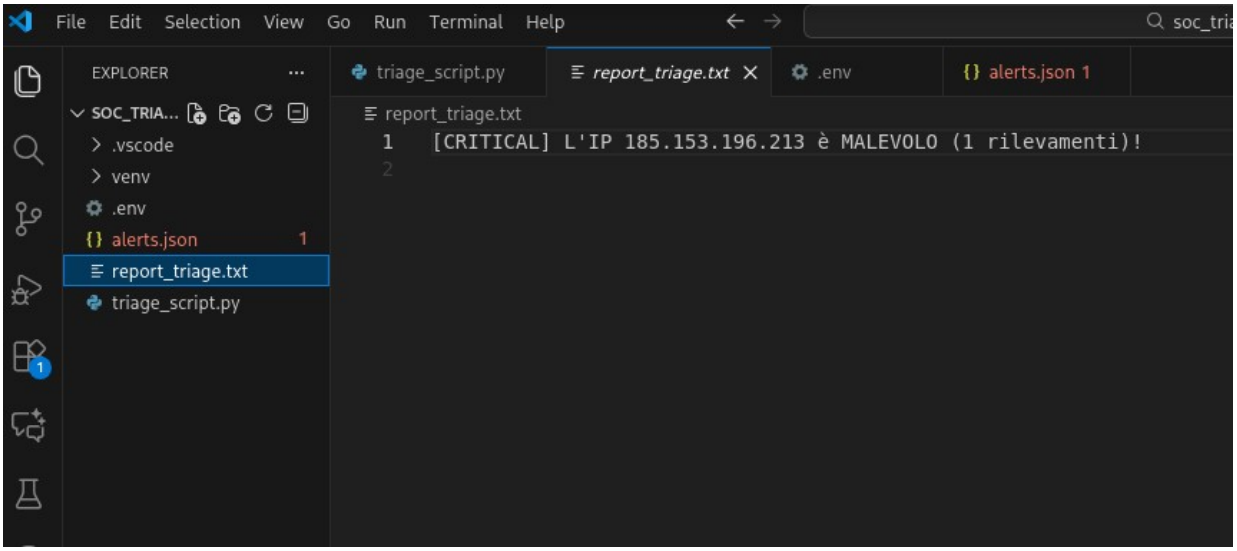

Fig. 6 - Storicizzazione dell'evento di sicurezza tramite scrittura in append sul file report_triage.txt.

4. Ottimizzazione e Funzionalità Avanzate

4.1 Implementazione del Caching Locale

Al fine di ottimizzare il consumo delle quote API (*rate-limiting*) e ridurre i tempi di latenza causati dall'overhead di rete, lo strumento è stato potenziato con un meccanismo di **caching in-memory**.

Sfruttando la struttura dati a dizionario di Python, lo script mappa e memorizza gli indirizzi IP già analizzati associandovi il relativo **Threat Score**.

Per ogni nuovo evento processato, il sistema verifica la presenza dell'indicatore nella cache: in caso affermativo, l'interrogazione esterna viene bypassata, attingendo direttamente al dato storicizzato.

4.2 Validazione e Gestione delle Eccezioni

Per verificare la resilienza dell'applicativo, lo script è stato testato contro un dataset "trappola" contenente log **NDJSON** malformati o privi di chiavi fondamentali (*es. "src_ip"*).

L'implementazione dell'Exception Handling ha intercettato con successo le anomalie, **prevenendo crash** applicativi e garantendo l'elaborazione continua in streaming. L'efficacia della cache in-memory è stata altresì confermata, in quanto l'applicativo ha bypassato le query API per gli indicatori duplicati.

```
┌──(venv)─(kali㉿kali)-[~/Desktop/soc_triage_project]
└─$ python triage_script.py

--- Analisi per: 8.8.8.8 ---
🌐 [API CALL] Risposta ricevuta e salvata in cache.
✅ [INFO] L'IP 8.8.8.8 risulta sicuro (0 rilevamenti).

--- Analisi per: 185.153.196.213 ---
🌐 [API CALL] Risposta ricevuta e salvata in cache.
🚨 [CRITICAL] L'IP 185.153.196.213 è MALEVOLO (1 rilevamenti)!

--- Analisi per: 185.153.196.213 ---
⚡ [CACHE HIT] Trovato in memoria! Salto la chiamata API.
🚨 [CRITICAL] L'IP 185.153.196.213 è MALEVOLO (1 rilevamenti)!
```

Fig. 7 - Validazione del meccanismo di caching locale: la prima occorrenza dell'indicatore malevolo innesca l'interrogazione dell'API (riquadro blu), mentre la successiva genera un "Cache Hit" attingendo alla memoria (riquadro arancione), azzerando l'overhead di rete.

4.3 Notifiche Real-Time via protocollo SMTP

Per minimizzare il Mean Time To Respond (MTTR) in caso di incidenti, il sistema di alerting è stato esteso introducendo notifiche push via **posta elettronica**.

Qualora il Threat Score superi la soglia di tolleranza (*rilevamenti > 0*), lo script si avvale della libreria nativa **smtplib** per generare ed inviare un avviso strutturato.

La comunicazione verso il mail server (es. relay Gmail) avviene tramite protocollo SMTP sulla porta **587**, forzando l'upgrade crittografico della sessione tramite **STARTTLS**.

Le credenziali di autenticazione (*App Password*) sono state rigorosamente segregate come variabili d'ambiente per scongiurare l'esposizione di segreti aziendali.

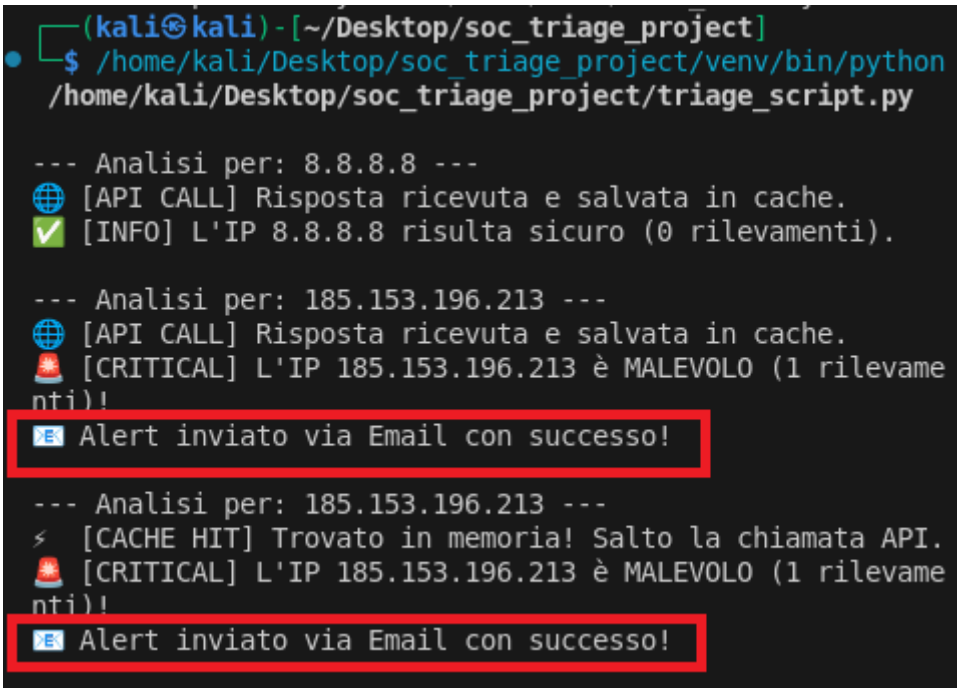

Fig. 8 – Invio della Mail di alert da parte del programma.

Fig. 8 – Esempio di Alert ricevuto via mail.

5. Conclusioni

Il progetto ha dimostrato con successo l'efficacia dell'automazione applicata al triage di primo livello in ambito SOC.

Lo strumento sviluppato azzera i tempi di analisi manuale dei log di rete, assicura la corretta gestione delle risorse di sistema tramite un approccio **Memory-safe I/O** e previene l'esposizione di credenziali integrando variabili d'ambiente.

L'arricchimento automatico tramite **Threat Intelligence** e l'implementazione di un sistema di alerting real-time via protocollo **SMTP** trasformano un log raw in "actionable intelligence", riducendo il **Mean Time To Respond** (*MTTR*) e innalzando drasticamente la postura di sicurezza aziendale.